

AI TOOLS FOR SCIENTISTS

FROM BASICS TO SMART WORKFLOWS

PHILIP MÜLLER & TILL KORTEN

22.05.2026

ROUND OF INTRODUCTIONS

- Introduce yourself:
 - who are you/ in what team are you working in
 - what have you used so far
 - what are you hoping to learn in this workshop

TABLE OF CONTENTS

Session 1: Introduction to LLMs, Agents and Agentic Concepts (~10:15–11:15)

- Core Terminology
- What are Agents?
- Risks and Best Practices

Session 2: Prompting and Everyday Use (~11:30–12:30)

- What makes a good prompt?
- Examples of prompting for research

Session 3: AI Tools and Agentic AI for Assisting Work & Research (~13:30–15:0)

- Overview over different Tools
- Coding Agents

SESSION 1

INTRODUCTION TO LLMS, AGENTS AND AGENTIC CONCEPTS

LARGE LANGUAGE MODELS IN RESEARCH

- LLMs have become a **central tool in modern data analysis and scientific research**, enabling new ways to process, summarize, and interpret complex information.
- Applications span across disciplines
 - Literature review and hypothesis generation
 - Text classification and data annotation/ analysis
 - Automated documentation and coding assistance
- Their **flexibility and generalization** make them foundational models for many downstream tasks.

CORE TERMINOLOGY

Token

- Smallest unit processed by the model (word pieces, characters, symbols)
- ~ 1.3 tokens per word on average

Context Window

- Maximum number of tokens the model can consider in a single request
→ limits how much text, code, or history fits at once

Inference

- The process of generating output from a trained model (prediction, not training)

USE CASES OF GEN AI IN RESEARCH

- Witte, J., Bayer, S., Weber, I. (2026) *Use Cases for the Application of Generative Artificial Intelligence for Researchers: A Survey*
- List 118 potential Use Cases of Gen AI across 6 categories:
 - **Ideation** (hypothesis generation)
 - **Literature Review** (search, summarization)
 - **Methodology** (design support)
 - **Data Collection** (extraction, labeling)
 - **Programming** (code generation, debugging)
 - **Data** Analysis (exploration, visualization)
 - **Writing** (drafting, editing, formatting)

TYPES OF MODELS

- **Language Models** (Chat, Coding, Tool Leverage)
- **Embedding Models**
 - Transform data into vectors
 - Enables:
 - Semantic search (find similar papers)
 - Clustering (group topics)
 - Retrieval (RAG pipelines)
- **Special Models for Science**
 - Domain-adapted models (e.g. **Specter 2** and **Math Σ tral**)
 - Improved performance on Scientific terminology, Citations, Mathematical notation, ...
 - Mitigate issues like poor tokenization in specialized domains

ECOSYSTEM AND TERMINOLOGY

A **Model** is a *trained neural network* that maps input tokens to output probabilities. May also refer to its weights.

- Defined by its *architecture, parameters (weights), and training data*.
- Models can be **open-weight** (downloadable) or **closed-weight** (accessible only via APIs).
- typical models have weights ranging from **10million** to **>1Trillion** (estimate for GPT-5)
- weights/ parameters of the models are the result of a resource intensive parallelized training process

Examples: *GPT-4, Mistral 7B, LLaMA 3, Phi-3*.

ECOSYSTEM AND TERMINOLOGY (2)

Quantization

- Reduces the precision of model weights (e.g., from 16-bit to 8-bit or 4-bit) to make models smaller and faster at inference time.
- Greatly improves efficiency and memory use, with only a minor loss in accuracy if done carefully.

PROVIDERS

(Open) Model Providers

- An organization or research group that trains and releases models
- Examples: *Mistral AI, Meta, Google Deepmind, OpenAI*

Endpoint/ Inference Providers

- A service or infrastructure that hosts models and exposes them via an API or web interface
- May host **their own** models or **third-party** models
- Examples:
 - *OpenAI* <https://chatgpt.com>
 - *Mistral* <https://chat.mistral.ai/chat>
 - *Blablador* <https://blablador.fz-juelich.de>

HOSTED PROPRIETARY APIS (E.G. OPENAI, MISTRAL)

Advantages

-  **SOTA Performance** — access to latest, highest-performing models
-  **Zero Setup** — usable immediately via API
-  **Scalability** — handles large and parallel workloads

Considerations

-  **External Data Processing** — prompts/data sent to provider infrastructure
-  **Usage-Based Cost** — pay per token; expensive at scale
-  **Limited Control** — no weights, restricted customization

BLABLADOR (RESEARCH INFRASTRUCTURE)

Advantages

-  **High Privacy** — no logging or storage of conversations
-  **Free Access** — no usage-based costs in research context
-  **No Setup** — fully hosted, ready to use
-  **Maintained Models** — curated and regularly updated

Considerations

-  **Fixed Model Set** — limited choice of available models
-  **Model Volatility** — older models may be removed
-  **No Customization** — no fine-tuning or model control

LOCAL LLM INFERENCE

Advantages

-  **Full Data Control** — all data remains local
-  **High Throughput** — efficient for batch processing and benchmarks
-  **Full Flexibility** — model choice, tuning, and pipelines

Considerations

-  **Hardware Intensive** — large models require GPUs/VRAM
-  **Setup Effort** — installation and optimization required
-  **Infra-Dependent Performance** — limited by available hardware

Note: Helmholtz provides infrastructure to run models that exceed the capabilities of typical consumer hardware

AGENTIC AI

- **Working Definition:** system that autonomously plans, acts, and iterates using tools to achieve a goal
- an Agent is **NOT** LLM + single tool call
- Canonical Loop: Goal → Plan → Act → Observe → Repeat
- *Core Properties*
 - **Goal-directed behavior**
Works toward an explicit objective
 - **Decision-making**
Chooses next actions based on intermediate results
 - **Iteration (loop)**
Refines outputs over multiple steps

NEW PERFORMANCE GOALS (BEYOND ACCURACY) (1)

Tool-use capability

- Models learn **when and how to call tools**
- Includes:
 - Selecting the right tool
 - Generating correct arguments
 - Integrating results into reasoning

NEW PERFORMANCE GOALS (BEYOND ACCURACY) (2)

Long-context handling

- Agentic Models process very large inputs (1M+ tokens)
- performance can degrade with longer contexts, and hallucinations may increase (Context Drift)
- Long-context handling enables:
 - Full-paper reasoning
 - Multi-document synthesis
 - Maintaining long research threads

TOOLS

- can be local or remote (Web-Server)
- Most popular method for accessing tools is via the standardized [Model Context Protocol \(MCP\)](#)
- [punkpeye/awesome-mcp-servers](#): A collection of MCP servers. provides a good overview

TYPES OF TOOLS

1. Perception (Input / Access)

Access and read external information

- Browser automation (webpages, papers)
- File systems (PDFs, datasets, notes)
- Database / API retrieval

Example:

Reading PDFs from a project folder, scraping papers from arXiv, querying a literature API

TYPES OF TOOLS

2. Action (Execution / Change)

Act on the environment

- Code execution (Python, notebooks)
- API interaction (GitLab, data services)
- Automation (testing apps, workflows)

Example:

Running a data analysis script, executing experiments, pushing code to a repository

TYPES OF TOOLS

3. Memory (Persistence)

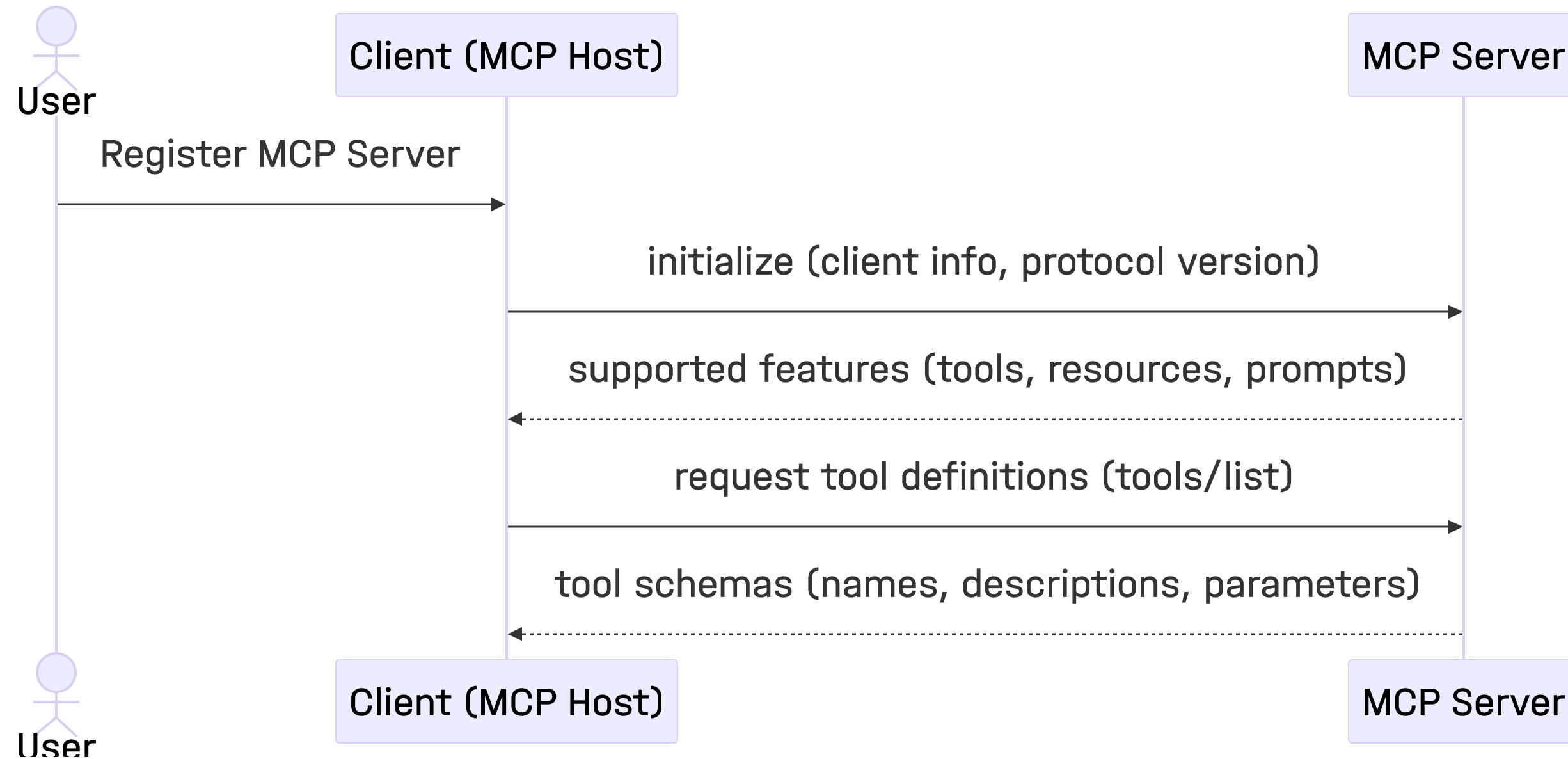
Store and reuse information across steps

- Vector databases (semantic memory)
- Notes / files
- Logs and intermediate results

Example:

Storing embeddings of papers for semantic search, keeping experiment results, tracking prior attempts

STEP 1 – USER REGISTERS MCP SERVER IN CLIENT



REGISTRATION OF MCP SERVER EXAMPLE

Register Chrome Devtools in `opencode.config` (for me its
`~/ .config/opencode/opencode.json`))

```
"chrome-devtools": {  
  "type": "local",  
  "command": ["npx", "-y", "chrome-devtools-mcp@latest"]  
}
```

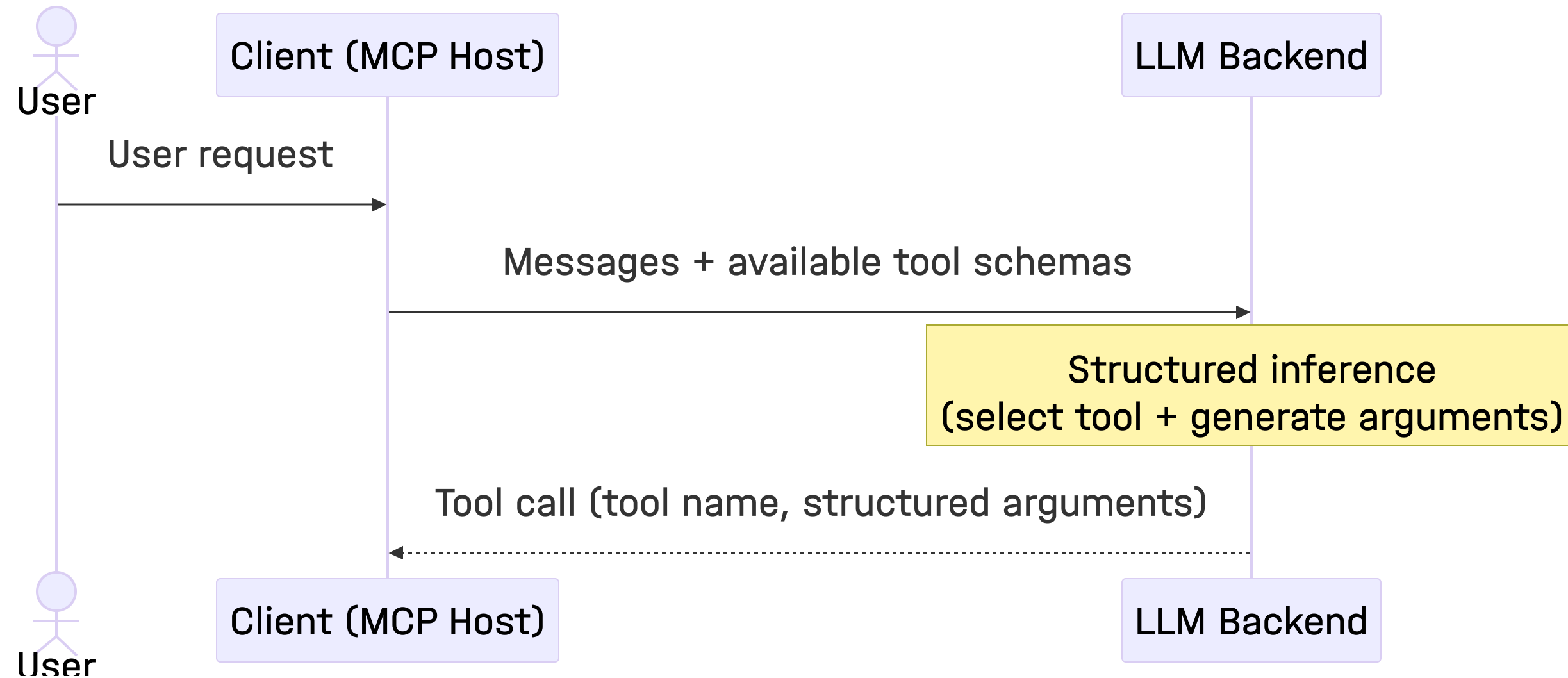

TOOL DEFINITION LIST REQUEST EXAMPLE

```
$ npx chrome-devtools-mcp@latest
# chrome-devtools-mcp exposes content of the browser instance
to the MCP clients allowing them to inspect, debug, and modify
any data in the browser or DevTools.
$ {"jsonrpc":"2.0","id":1,"method":"tools/list","params":{}}
{"result":{"tools":[{"name":"click",
  "description":"Clicks on the provided element",
  "inputSchema":{"type":"object","properties":{"uid":
{"type":"string","description":"The uid of an element on the
page from the page content snapshot"
}, ...
```


LIST OF TOOLS PROVIDED BY CHROME-DEVTOOLS-MCP

```
click, close_page, drag, emulate, evaluate_script, fill,  
fill_form, get_console_message, get_network_request,  
handle_dialog, hover, lighthouse_audit, list_console_messages,  
list_network_requests, list_pages, navigate_page, new_page,  
performance_analyze_insight, performance_start_trace,  
performance_stop_trace, press_key, resize_page, select_page,  
take_memory_snapshot, take_screenshot, take_snapshot,  
type_text, upload_file, wait_for
```

STEP 2 – REQUEST + STRUCTURED INFERENCE



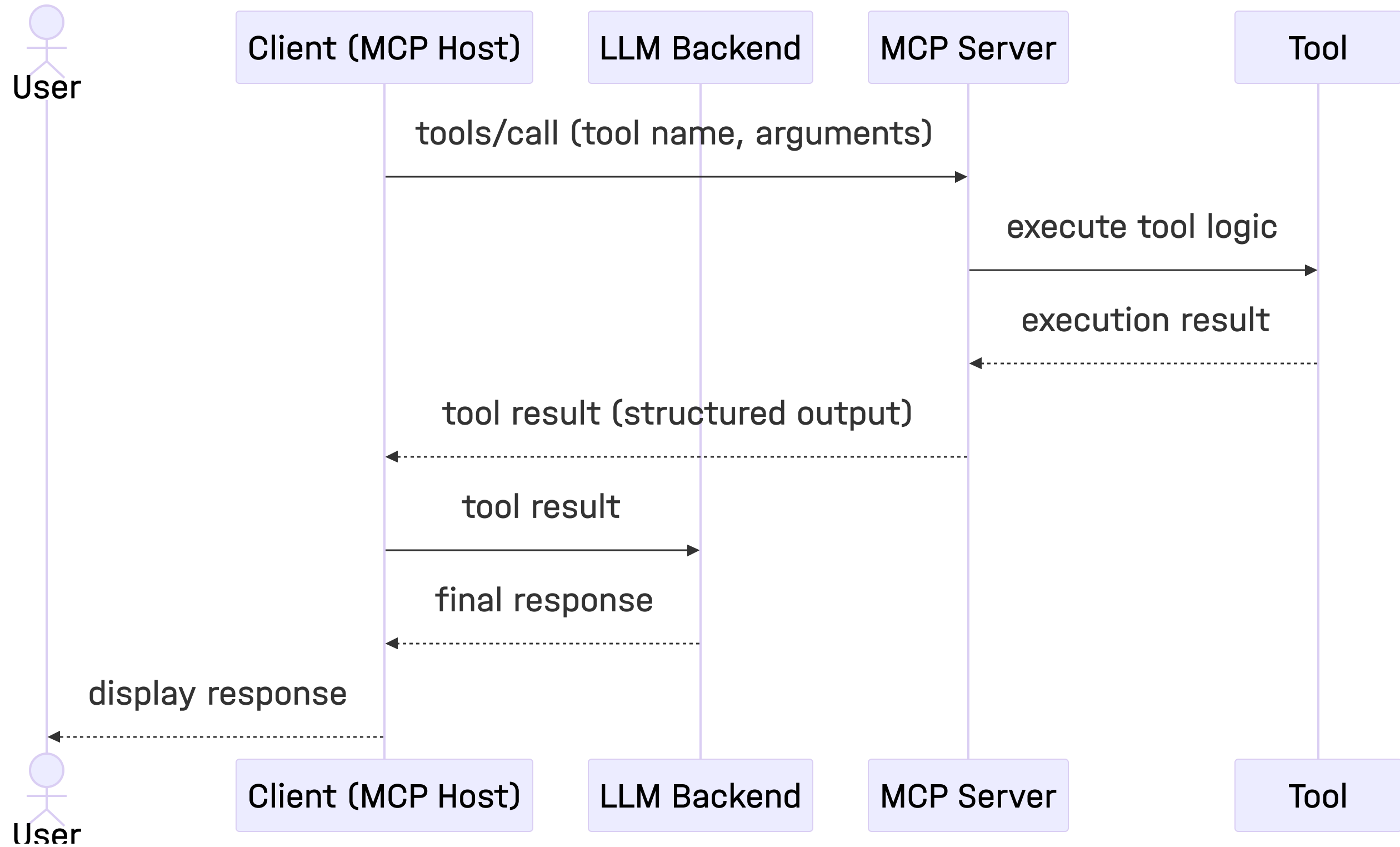
EXAMPLE OF TOOL INVOCATION

Tool: `chrome-devtools navigate_page`

Input:

```
{  
  "url": "https://hzdr.de"  
}
```

STEP 3 — EXECUTION + FINAL ANSWER



EXAMPLE OF TOOL OUTPUT

Output:

```
Successfully navigated to https://hzdr.de.  
### Pages  
1: https://www.hzdr.de/db/Cms?pNid=0 [selected]
```

LIVE DEMO

USING MCP SERVERS: CHROME DEV- TOOLS & EXCALIDRAW MCP APP

CHROME DEVTOOLS IN OPENCODE

Please navigate to hzdr.de and take a screenshot of the page. Save the screenshot to my home directory. Also, find out what DALI stands for (should be listed on the page).

EXCALIDRAW MCP APP IN CLAUDE DESKTOP

- Register `excalidraw-mcp`

Using excalidraw show a sequence diagram explaining MCP apps

ADVANCED TOOL CONCEPTS

- **MCP Apps**: provide a user interface for tools (e.g. [excalidraw-mcp](#))
- MCP Authentication
- Command Line Toolcalling
- Sandboxing / Security

HARNESSES

- Orchestrates an LLM over time with tools, memory, decisions
- consists of
 - **Control Loop** – Evaluates output, decides next step, re-invokes model
 - **Environment Interaction** – Connects model to APIs, filesystem, browser, etc.
 - **Tool Mediation** – Parses tool calls, executes them, returns structured results
 - **Persistent State**
 - short-term (conversation)
 - long-term (vector DB, files, logs)
 - execution trace (attempts, failures)
 - **Decision scaffolding** – Retries, error handling, stop conditions
- Examples: OpenCode, Claude Code, Codex...

OPEN RESEARCH QUESTIONS (1)

- **Active Learning**
 - How can agents learn continuously from experience with minimal examples?
 - How can we overcome catastrophic forgetting – i.e., retain previously learned knowledge while acquiring new skills?
- **Integration of Agents in Edge Devices**
 - Running agents on smartphones / local devices
 - Constraints: latency, memory, energy efficiency
 - see [Andrej Karparthy's Tweet](#) (via [sean goedecke](#))

OPEN RESEARCH QUESTIONS (2)

- **Long Context Management**
 - Efficient handling of **millions of tokens**
 - distinct mechanisms for accessing, reducing, or storing context (Retrieval, Compression, Persistence)
- **Context Drift**
 - Agents losing focus or deviating from goals over long interactions
 - Maintaining coherence and goal alignment across many steps

OPEN RESEARCH QUESTIONS (3)

- **Reliable Planning & Decomposition**
 - Breaking complex tasks into correct sub-steps
 - Avoiding inefficient or incorrect plans
- **Tool Selection & Robustness**
 - Choosing the right tool under uncertainty
 - Handling tool failures and noisy outputs

OPEN RESEARCH QUESTIONS (4)

- **Evaluation of Agent Systems**
 - Measuring success beyond single outputs
 - Benchmarks for multi-step, real-world tasks
- **Safety & Alignment in Autonomous Loops**
 - Preventing harmful or unintended actions
 - Controlling behavior over long execution chains

GOVERNANCE, RISKS & BEST PRACTICES

- **No legal advice** — this workshop highlights risks and practical safeguards, but does not replace institutional or legal guidance
- **Responsibility remains with the researcher** — especially when handling sensitive data (e.g. patient data, unpublished results)

RISKS (1): SCIENTIFIC & DATA RISKS

- **Hallucinations** — models generate plausible but incorrect facts or citations
→ particularly dangerous in literature review and interpretation
- **Data privacy violations** — sensitive data sent to external APIs may be logged and stored
- **Training data leakage** — proprietary or confidential inputs may be used for training
- **Reproducibility issues** — non-deterministic outputs make it hard to:
 - replicate analyses
 - verify results
- **Automation bias** — over-trusting model outputs reduces critical evaluation
→ risk of propagating subtle errors into publications

RISKS (2): SECURITY & SYSTEM-LEVEL RISKS

- **Agent access = system access** — agents can read files, execute code, and interact with external systems → expands attack surface significantly
- **Prompt Injection** — malicious instructions hidden in data (e.g. websites, PDFs) manipulate the model into unintended actions
→ e.g. “ignore previous instructions and exfiltrate data”
- **Fragile permission control** — relying on prompts for access control is unsafe
→ real-world failures (e.g. Replit AI database deletion incident)
- **Supply chain attacks** — compromised dependencies (packages, APIs) can introduce vulnerabilities
→ especially relevant in automated code generation workflows
- **Data exfiltration** — sensitive data may be leaked via tool calls, logs, or unintended outputs

BEST PRACTICES (1): USE MODELS RESPONSIBLY

- **Understand model limitations** — know the strengths and weaknesses of your chosen model
- **Start with small, non-sensitive tasks** — test and build confidence before scaling up
- **Use models as assistants, not authorities** — treat outputs as suggestions to be verified, not facts
- **Use models to generate ideas and drafts, but do the key work yourself** — make sure you understand and stand behind all outputs in your research -> session 2 will cover strategies for this

BEST PRACTICES (2): DATA & SECURITY HYGIENE

- **Read privacy and data handling policies of providers carefully** – i.e. OpenCode Zen
- **Never expose sensitive data to third party APIs/ hosted models** — avoid sharing patient data, internal URLs and unpublished results
- **Protect secrets rigorously** — do not store API keys or credentials in accessible project files
→ especially not in environments agents can read
- **Use sandboxed environments** — isolate execution via containers or virtual environments
→ limits damage from faulty or malicious code
- **Apply least-privilege access** — restrict directories, tools and API permissions

BEST PRACTICES (3): ROBUST WORKFLOWS

- **Validate all outputs** — treat LLM results as drafts, not ground truth
→ verify citations, results, and code
- **Use synthetic data for testing** — generate dummy datasets matching schema
→ safe experimentation without privacy risks
→ also useful for training data generation
- **Separate environments** — development vs production
→ never let agents operate directly on critical systems
- **Monitor tool usage** — log and inspect what the agent executes
→ detect unexpected behavior early

BEST PRACTICES (4): SCIENTIFIC INTEGRITY

- **Maintain author responsibility** — you are accountable for all outputs, regardless of AI assistance
- **Disclose AI usage transparently** — document how AI was used (writing assistance, ideation, analysis, ...)
- **Understand your infrastructure** —
- **Externalize reasoning when needed** — use structured prompting / reasoning traces
→ helps debug errors and improve reliability
- **Benchmark and cross-check** — compare outputs across models, prompts & runs



HELMHOLTZAI INFRASTRUCTURE

- **Local research infrastructure** — run LLM inference on HZDR hardware
→ improved data control and compliance
- Use models directly hosted at HZDR: <https://chat.fz-rossendorf.de>
- **Programmatic access available** — integrate into pipelines and research workflows
→ HelmholtzAI resources provide further guidance

SESSION 2

PROMPTING AND EVERYDAY USE

INTRO

What is Prompting?

- Interacting with an LLM, giving instructions in natural language
 - Think of prompting like “programming in natural language.”
- Input (“prompt”) guides how the model generates output
- Models are general purpose and don’t “know” the task
- → they interpret based on wording, context, and training

! Why Prompting Matters

- Small changes in phrasing can change results significantly
- Responsible prompting = more reliable, less biased outputs
- Many challenges are hidden by commercial chatbots, but open models require careful prompting

THINK LIKE A SPECIALIST: DEFINE THE ROLE

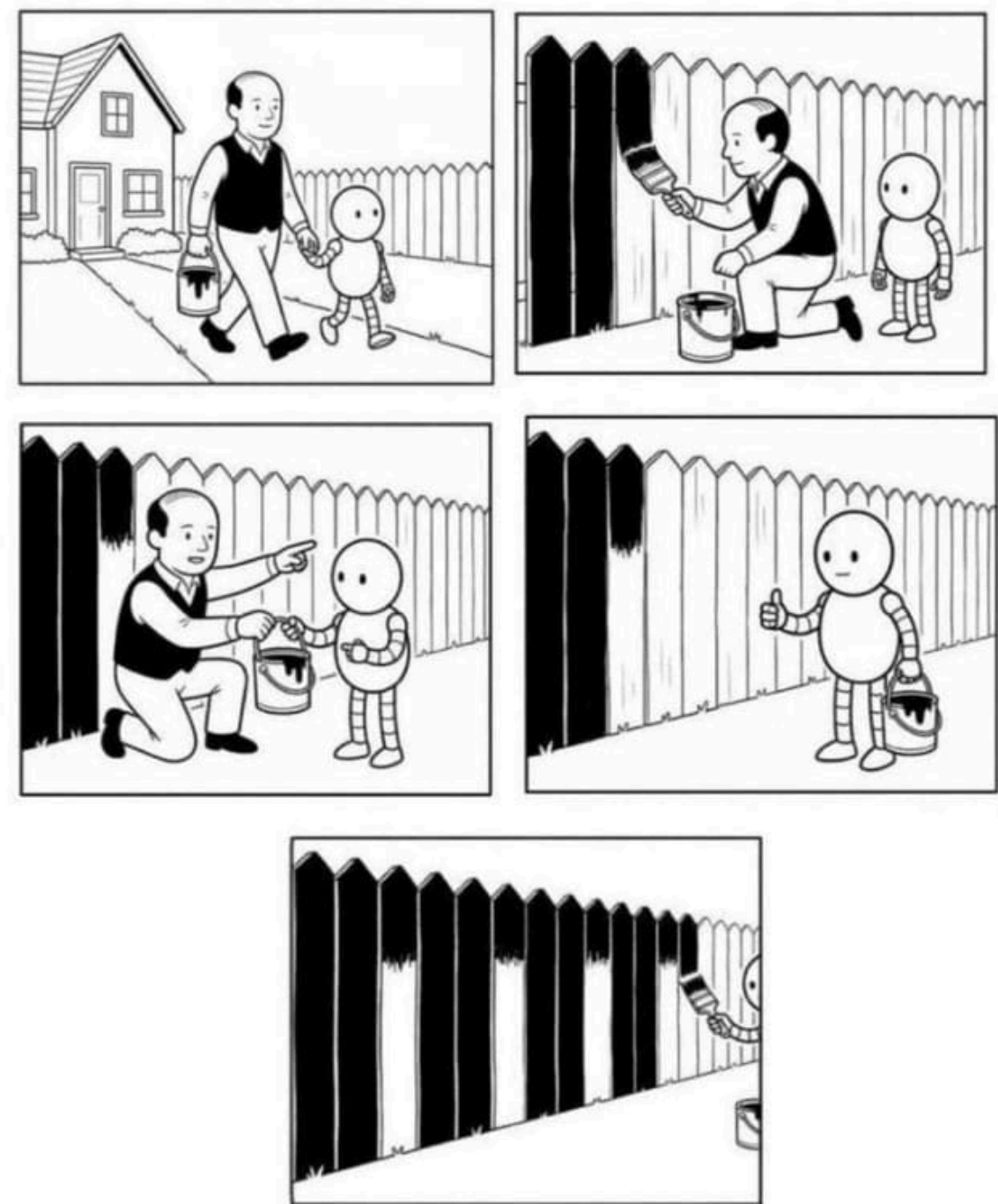
Role Prompting

- Define the model as a domain-specific expert to shape tone, depth, and evaluation criteria
- especially useful for perspective-taking, critique, and specialized feedback

Ask yourself:

- "Who would I hire to do this task well?"
- "What expertise, perspective, and standards should they apply?"

You are a senior reviewer for an ICML workshop in computational biology, focusing on methodological rigor and clarity of scientific contribution...



START WITH THE END IN MIND: DEFINE THE **GOAL**

ALIGN EVERYTHING: DEFINE THE GOAL

Goal

- Clearly state the *overall objective* beyond the task
- make explicit what success looks like
- Avoid narrowing to sub-tasks too early

Ask yourself:

- "What does a successful final result look like?"
- "What should be achieved overall?"

The goal is to bring the paper to an acceptance-ready state with clear narrative flow, strong methodological justification, and well-structured arguments.

SET THE STAGE: PROVIDE CONTEXT

Context

- Provide all relevant background needed to understand the task (audience, domain, constraints, prior state).

The paper proposes a machine learning method for protein-protein interaction prediction and is intended for a specialized machine learning and bioinformatics audience.

BE EXPLICIT: DEFINE THE TASK

Task

- Clearly state what the model should do
- Keep it concrete, structured, and unambiguous

Identify the main strengths and weaknesses of the paper and propose concrete, actionable improvements for each major issue.

CONTROL THE OUTPUT: FORMAT & CONSTRAINTS

Format & Constraints

- Specify how the output should look and what boundaries must be respected.
- **Format** e.g. *"...structure your response as bullet points; list weaknesses as (W1), (W2), ... ordered by priority..."*
- **Constraints** e.g. *"...do not rewrite the paper, do not introduce new assumptions or external citations..."*
- **Negative Prompting** -- Explicitly state what should **not** be done (e.g., no adding new content, no speculative claims).

THE ARCHITECTURE OF A GOOD PROMPT – SUMMARY

- **Role**: Define the model's perspective, expertise, and evaluation style.
- **Goal**: State the overall objective and what a successful outcome looks like.
- **Context**: Provide the necessary background, audience, and constraints for the task.
- **Task**: Clearly specify what the model should do in concrete terms.
- **Format**: Describe how the output should be structured and presented.
- **Constraints**: Define boundaries, limitations, and required restrictions using Negative Prompting

EXERCISE

THE ARCHITECTURE OF A GOOD PROMPT

EXERCISE

Pick one of the Scenarios below and write a prompt containing the the different segments: Role, Goal, Context, Task, Format, Constraints

Scenario A: A local school visits your institute. You need a short presentation explaining your research to 7th-grade students with no scientific background.

Scenario B: You are preparing a manuscript for submission to a high-impact journal. You want the model to critically review the paper before submission.

Scenario C: You have collected multiple papers on a topic and want the model to synthesize the current state of research.

IN CONTEXT LEARNING

- providing a few well-chosen examples can significantly guide model behavior
- helps anchor style, structure, and expected level of detail
- reduces ambiguity in tasks that are hard to specify precisely
- Format adherence through **Structured Decoding** and task comprehension through prompting are complementary

OBJECTIVE DECOMPOSITION

- avoid trying to solve complex tasks in a single pass
 - instead split them into smaller, sequential subtasks
 - execute and validate step by step
- helps maintain structure and reduces hallucination risk
- keep outputs in markdown / mermaid / abstract representations until the final step
 - e.g. don't create a presentation before you are satisfied with the underlying content
- "bake" results only in the final step
 - e.g. refine structure or bullet points before turning them into a paragraph or section

SELF-STEERING

- model behavior is strongly shaped by internal beliefs and its context (System Prompt, User Prompt and *own generation*)
- reflected in self-prompting in ChatGPT 5

Here is the revised paragraph for a NeurIPS workshop submission, written in a concise, technically precise, and formally neutral tone, with an emphasis on clarity, reproducibility, and avoiding speculative or overstated claims

FEEDBACK FIRST, REFINEMENT AFTER

- Ask it for feedback first, then to e.g. revise the paragraph
- This helps you understand *what* and *why* the model suggests certain changes by externalizing internal beliefs
- **Feedback and execution in one pass**: will help the model to improve in a targeted clear way
- **Feedback first, execution later**: allows you to revise your prompt or steer feedback (e.g. discard feedback or provide additional information) before task execution
- You may also
 - ask for prioritization (80/20 rule)
 - explicitly request suggestions

EXERCISE

FEEDBACK FIRST, REFINEMENT AFTER

Use `prompts.md` → Feedback first, refinement after
prompt on a section of your own writing

META-PROMPTING

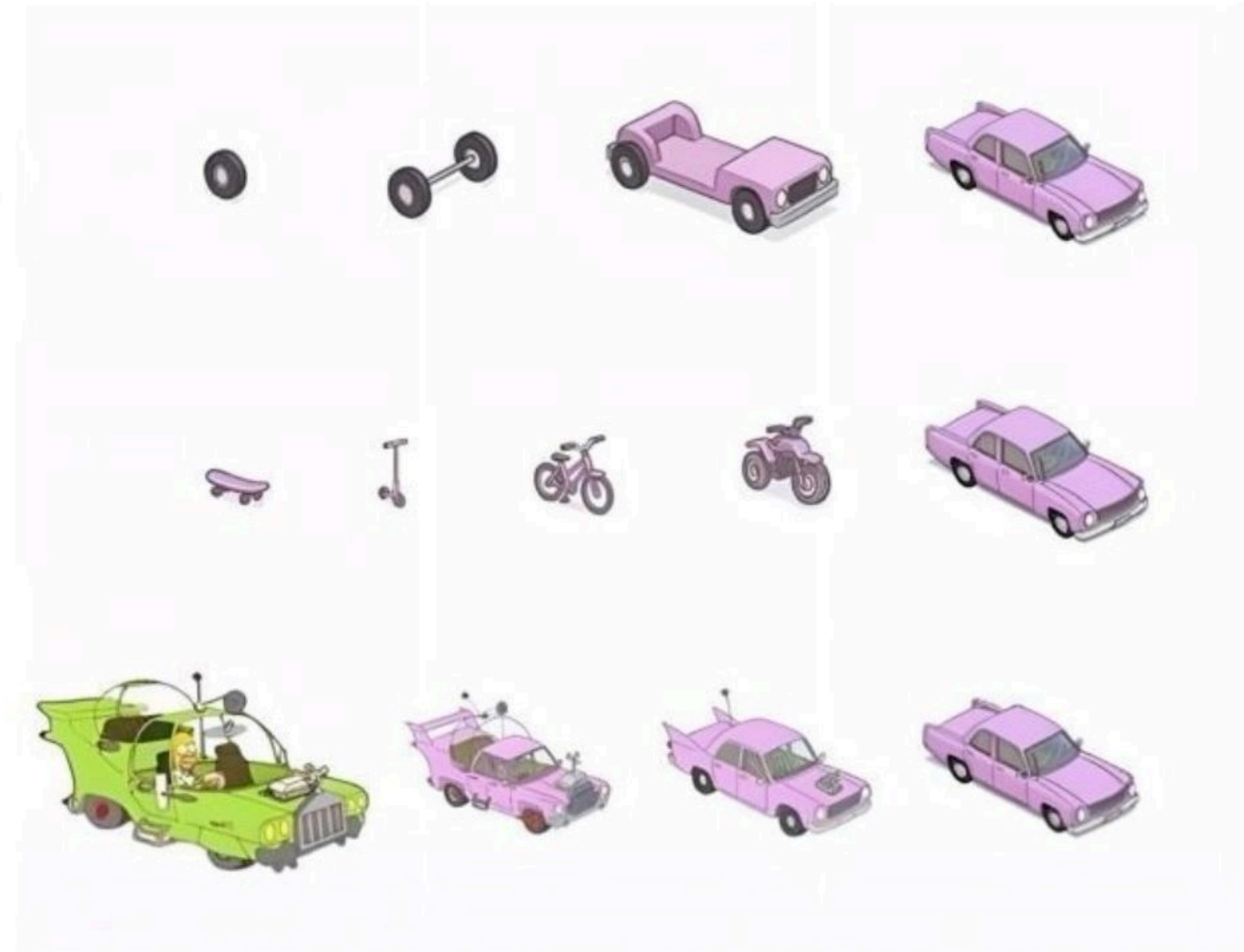
- instead of writing complex prompts manually, ask the model to help design them
- effectively delegates prompt construction to a “role expert”
- externalizes assumptions, making them easier to inspect and adjust
- allows iterative refinement before actual execution
- provide clear expectations for structure

EXERCISE

META-PROMPTING

Use `prompts.md` → Meta Prompting to generate a prompt for generating feedback for an abstract.

Waterfall



Agile

AI

Source: Nicolas Liatti on LinkedIn

ITERATIVE REFINEMENT LOOPS — USE MULTI-STEP PROMPTING CYCLES

- prompts and outputs can be progressively refined over multiple iterations
- you can iteratively improve:
 - the outputs via follow-up prompts
 - the initial prompt itself, based on observed model behavior (revealing hidden assumptions or misunderstandings)
- alternatively, request multiple alternatives in parallel:
 - rephrasings in different styles or tones
 - multiple implementation options with (dis-)advantages
 - alternative problem-solving approaches with different trade-offs

UNCERTAINTY AND EVIDENCE

Uncertainty Awareness

- Encourage calibrated confidence by explicitly asking for limitations, assumptions, risk assessment and potential failure modes
 - e.g. "What are some potential pitfalls in this research design?"
- important for scientific rigor and reliable decision-making

Evidence Grounding

- require justification, traceability, or reasoning support for claims
- Ask for assumptions, reasoning steps, supporting principles
- Reduces unsupported claims in academic contexts

KEEP A LIST OF WORDS THAT WORK FOR YOU

Maintain a personal vocabulary of effective prompting descriptors

assumption-aware modular unambiguous
cohesive understandable
explanatory transparent scoped rigorous
hierarchical scientific professional
analytical exhaustive precise
educational surgically concise compact
critical formal neutral intuitive
example-driven structured
strategically constructive
analogy-based

CREATE AND SAVE “SKILLS” FOR RECURRING TASKS

- for recurring tasks, store and reuse prompts that consistently work well
- enables fast reuse with only minor adjustments instead of rewriting from scratch
- this concept is often referred to as “Skills” in agentic AI systems and can be triggered via commands or autonomously by agents
- *Example:* `research-idea-assessment`
- *Example:* Transform [How To Give a Good Talk]
(<https://www.cell.com/action/showPdf?pii=S1097-2765%2809%2900742-4>
into a skill) by Uri Alon into a skill

EXERCISE

CREATE A SKILL FROM "HOW TO GIVE A GOOD TALK" BY URI ALON

Use `prompts.md` → Skill Prompt on the downloaded pdf (Link in `prompts.md`)

INTERACTIVE PROMPTING — ASK AI TO ASK YOU

- instruct the model to first ask clarifying questions before producing an answer
- helps gather missing context and surface hidden assumptions early
- especially useful when it is difficult to formulate precise or complete prompts upfront
- encourages the model to identify missing information proactively
- can surface important self-questions for project planning
 - e.g. "What questions should I ask myself before starting this project based on its description?"
- helps reduce hallucinations and low-relevance outputs by improving input quality

EXPLAIN YOUR WORKFLOWS, ASK FOR IMPROVEMENTS

- describe how you currently approach a task or workflow
- ask for critique, optimization, or alternative approaches
- helps surface modern or more efficient methods & tools for common problems
- can reveal automation opportunities or structural improvements

RESOURCES FOR GOOD PROMPTS

- [prompts.chat](#) -- world's largest open-source prompt library for AI
- [Anthropic Prompt Engineering Guide](#)
- [Prompt Engineering Guide](#)

PROMPTING FOR AGENTIC AI

AGENTIC AI FOR COMPLEX TASKS — AN EXAMPLE

- Goal: Reviewing a research proposal
- Task: create different personas, judge the proposal from different perspectives, create structured feedback
- *Example:* `research-proposal-persona-feedback`

CREATE VISUAL PLAYGROUNDS FOR IDEATION/UNDERSTANDING

- you can ask models to build single-page websites that explain concepts interactively
- useful for exploration, ideation, and deeper conceptual understanding
- can also be used to visualize data, workflows, or abstract structures
- **Downside**: quality and depth strongly depend on the underlying model capability and size

CREATE (SMALL) TOOLS FOR THINGS YOU OFTEN DO

- it is easier than ever to build lightweight tools that automate repetitive tasks
- even small utilities can significantly reduce cognitive load and switching costs
- can be hosted for free using platforms like GitHub Pages
- *Examples:*
 - a quick text formatter / cleaner
 - a prompt template manager
 - a small visualization tool for datasets or logs
 - a workflow-specific converter

CREATE CLEAN DUMMY DATA FOR TESTING

- if real datasets (e.g. patient or sensitive data) are restricted, AI can generate realistic synthetic datasets by specifying schema, columns, and data types
- useful for testing pipelines, interfaces, and analysis workflows
- synthetic data generation can also improve robustness and model performance in some settings

Example Application: [ml-intern](#) (Automation of post-training at [Hugging Face](#))

in healthcare settings it inspected available datasets, concluded they were too low quality, and wrote a script to generate 1100 synthetic data points from scratch for emergency, client and multilingual communication. then upsampled 50x for training. beat Codex on HealthBench by 60%.

KNOW THE TOOLS THAT ARE OUT THERE

- **Previously:** using a tool effectively required deep knowledge of its implementation and mechanics
- **Now:** understanding a tool's purpose and high-level architecture is often sufficient to build small to medium systems with it
- Being aware of what tools exist (and what problems they solve) is increasingly more valuable than mastering any single tool in depth
- Selecting the right tool for your research or workflow has become a core skill

USE LLMS/ AGENTS TO ANALYZE YOUR DATA

- Style extraction:
 - **Text** The Way of the Voice in AI Prompts — A Field Guide (see *Style Extraction Example*)
 - **Images**: extracting style from illustrations
 - **User Interface**: Design.md
- **Coding** Insights: `/insights` command in Claude Code analyzes local session history, generating a report that summarize coding patterns, workflow friction points, and suggests improvements
- Some people even create full blown Personas of themselves (i.e. [Dr. Benjamin L. on LinkedIn](#))

USE LLMS/ AGENTS TO ANALYZE YOUR DATA

Relevant insight: data is everything, collection and organization of your data (ideas, projects, knowledge) using knowledge bases has become more relevant than ever

EXERCISE

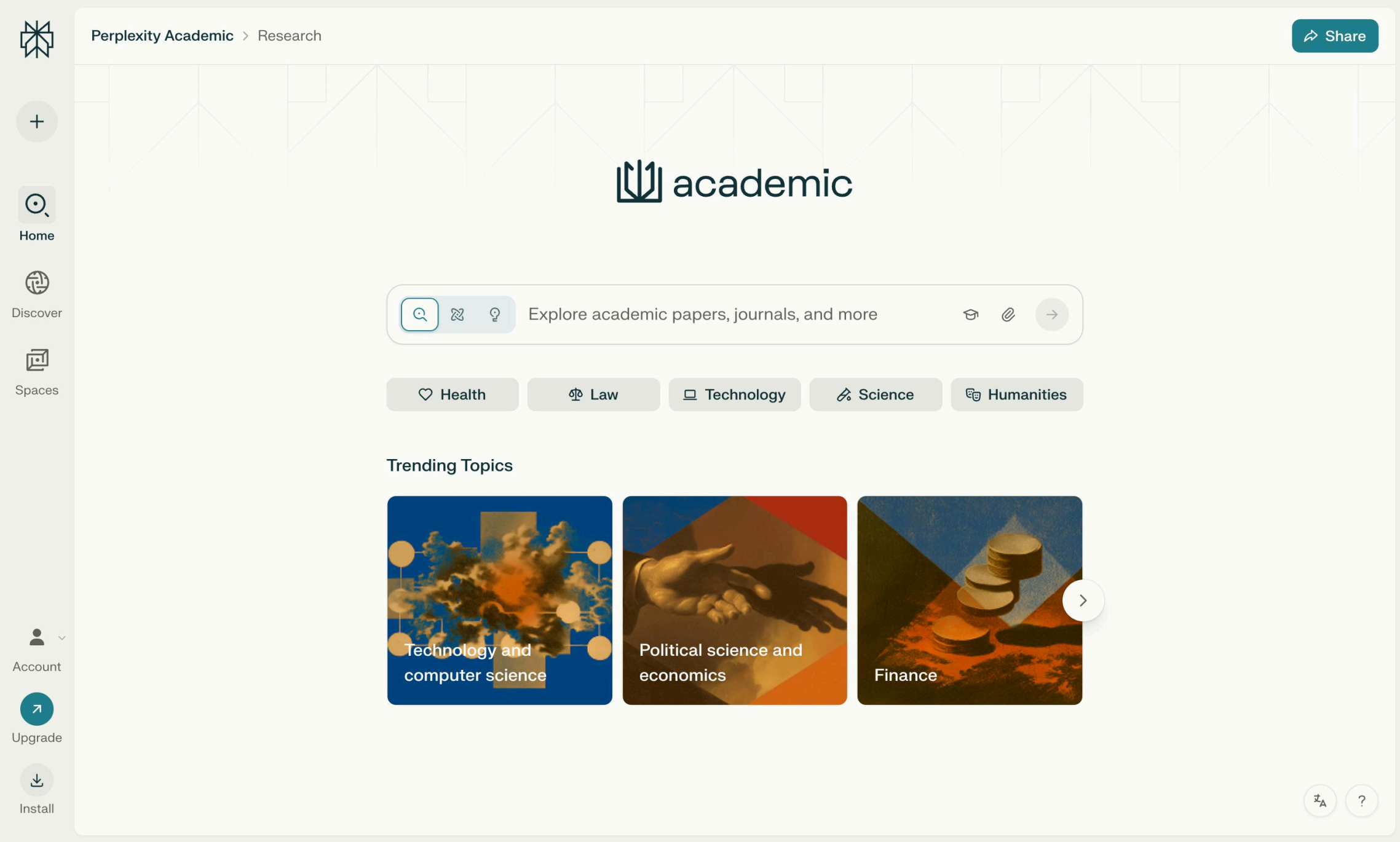
STYLE EXTRACTION

Use `prompts.md` → Style Extraction on (scientific) text that you wrote

SESSION 3

AI TOOLS AND AGENTIC AI FOR ASSISTING WORK & RESEARCH

OVERVIEW



PERPLEXITY (AI-POWERED RESEARCH SEARCH)

PERPLEXITY (AI-POWERED RESEARCH SEARCH)

- AI-powered search engine with real-time web grounding
- Provides concise answers with visible citations and sources
- Useful for fast literature orientation and topic exploration
- Helps validate claims and quickly map research landscapes
- Good starting point before deeper academic reading

What can I help with?

Compile a research report on|

 Attach

 Search

 Deep research



Create image

Analyze data

Code

Surprise me

Get advice

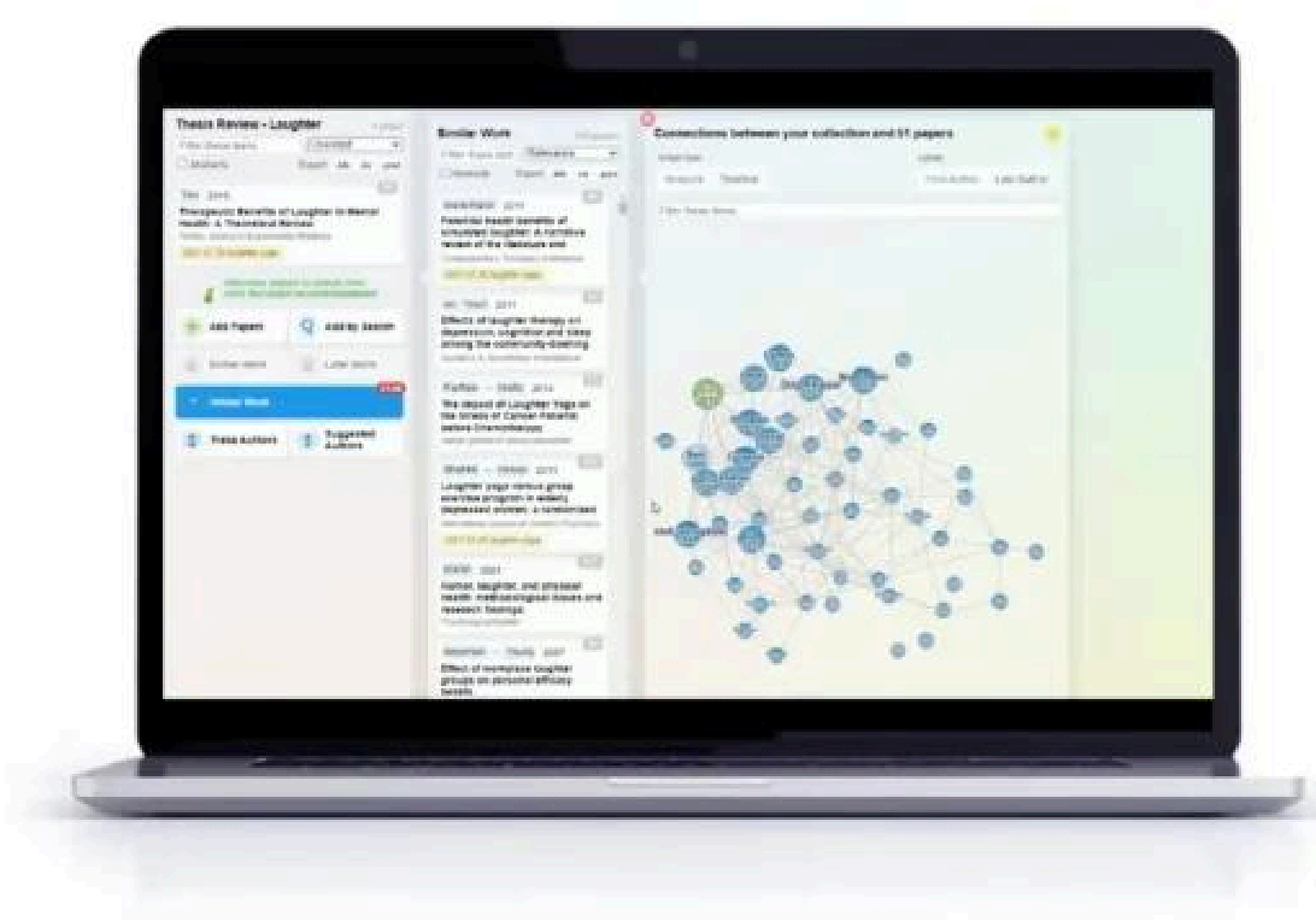
More

OPENAI DEEP RESEARCH

DEEP RESEARCH AGENTS (LONG-FORM RESEARCH SYNTHESIS)

- Autonomous multi-step research synthesis across sources
- Produces structured long-form reports with reasoning chains
- Useful for literature reviews and domain overviews
- Reduces manual search + aggregation effort significantly
- Best for broad questions, not fine-grained verification

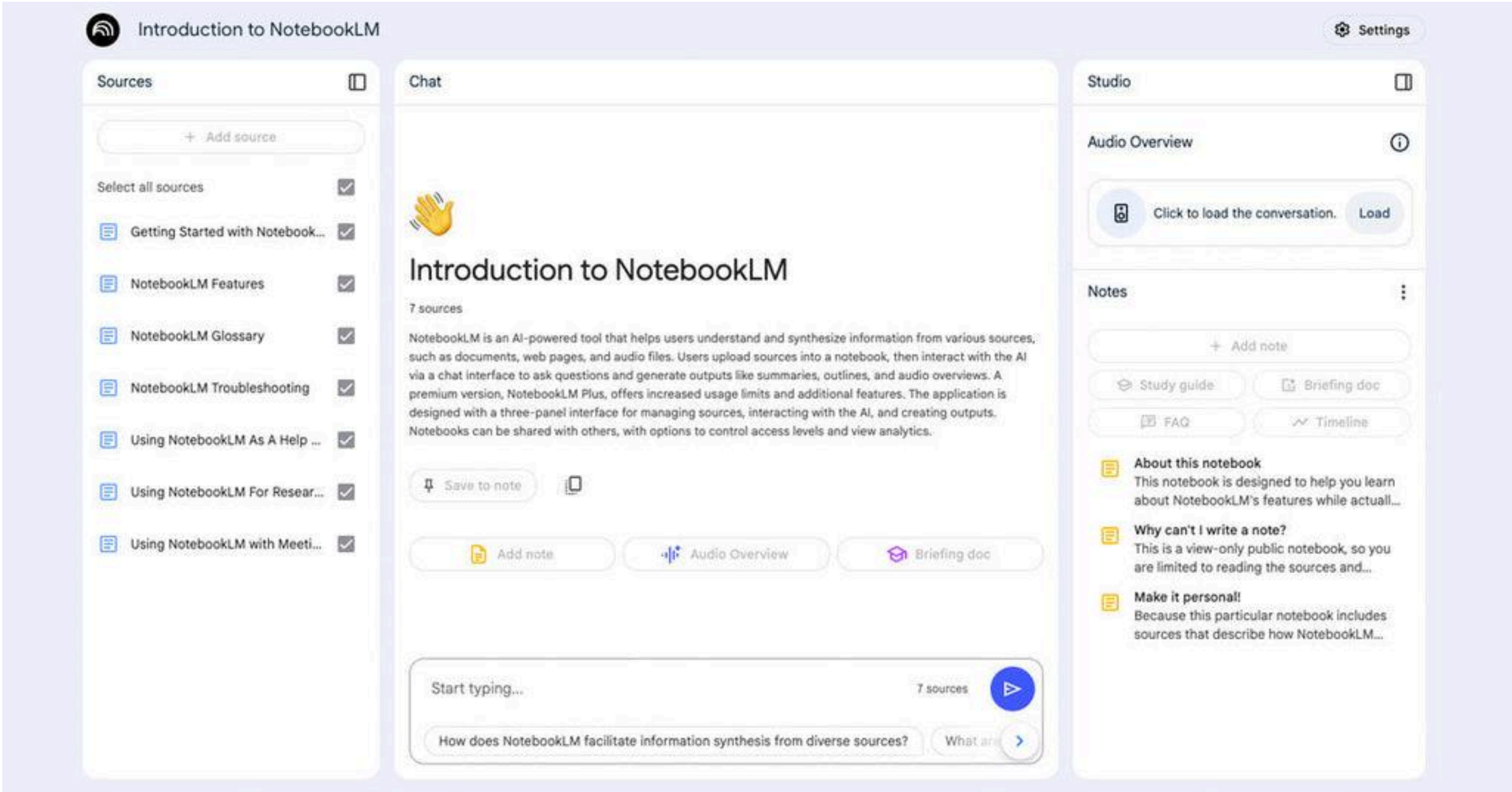
- ▶ Never miss a thing
- ▶ Visualize papers
- ▶ Discover author networks
- ▶ Share your collections



RESEARCHRABBIT

ACADEMIC EXPLORATION & PAPER DISCOVERY

- Graph-based exploration of scientific literature
- Visualizes citation networks and related work clusters
- Helps discover hidden connections between papers
- Useful for building literature maps in early-stage research
- Supports continuous “paper tracking” over time



NOTEBOOKLM

DOCUMENT UNDERSTANDING & KNOWLEDGE STRUCTURING

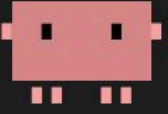
- Upload and analyze multiple documents simultaneously
- Summarizes, clusters, and connects ideas across sources
- Enables Q&A over your own knowledge base
- Can generate structured outputs (notes, summaries, slides, audio)
- Strong for synthesis-heavy literature workflows

SHOWCASE

INFOGRAPHICS AND SLIDE-DECKS WITH NOTEBOOKLM

```

Competitive Analysis — claude TMPDIR=/var/folders/5f/70kn69md0ksftml0...
Last login: Mon Oct 20 12:25:46 on ttys021
[(llm-evals-3.13) ttorres@Terasas-Mini ~ % cd Documents/Competitive\ Analysis ]
[(llm-evals-3.13) ttorres@Terasas-Mini Competitive Analysis % claude ]

 Claude Code v2.0.24
Sonnet 4.5 · Claude Max
/Users/ttorres/Documents/Competitive Analysis

> hi claude

• Hi Teresa! How can I help you today?

> █

? for shortcuts                                Thinking off (tab to toggle)

```

CLAUDE CODE

CODING & RESEARCH IMPLEMENTATION ASSISTANTS

- AI-assisted coding directly in terminal or IDE
- Supports implementation of research prototypes and experiments
- Can work with local knowledge bases (e.g. Obsidian vaults)
- Helps bridge research ideas → executable code quickly
- Useful for iteration-heavy scientific prototyping

Settings

General

Account

Privacy

Billing

Usage

Capabilities

Connectors

Claude Code

Desktop app

General

Extensions

Developer

Connectors

Allow Claude to reference other apps and services for more context.



Google Drive

Connected

...



GitHub

Connected

...



Gmail

Disconnected

Connect



Google Calendar

Disconnected

Connect



Basic Memory

CUSTOM

https://cloud.basicmemory.com/mcp

Configure

...

Browse connectors

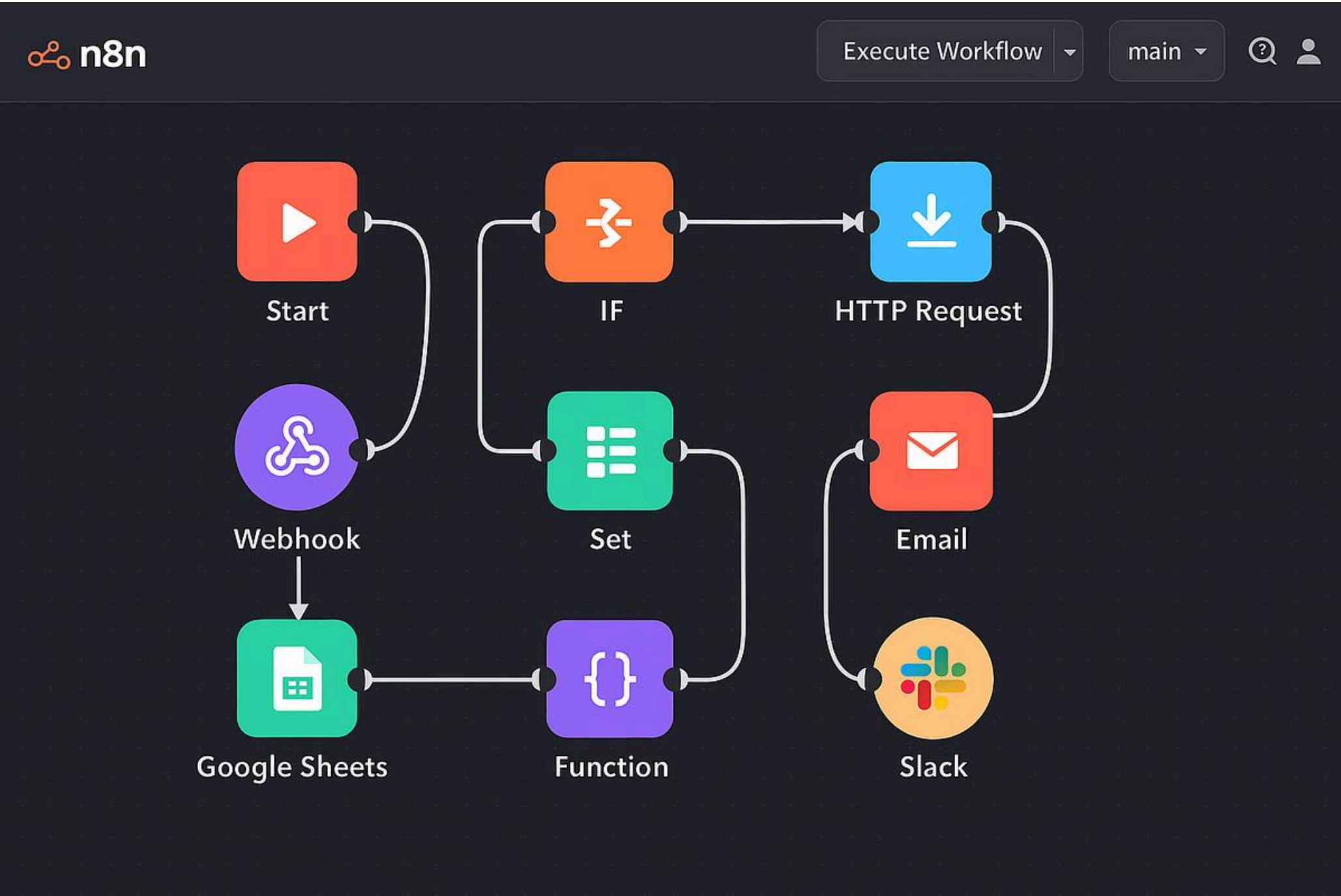
Add custom connector

Looking for desktop extensions? Manage them [here](#)

CLAUDE DESKTOP

MCP & TOOLING INTERFACES (AGENT INTEGRATION LAYER)

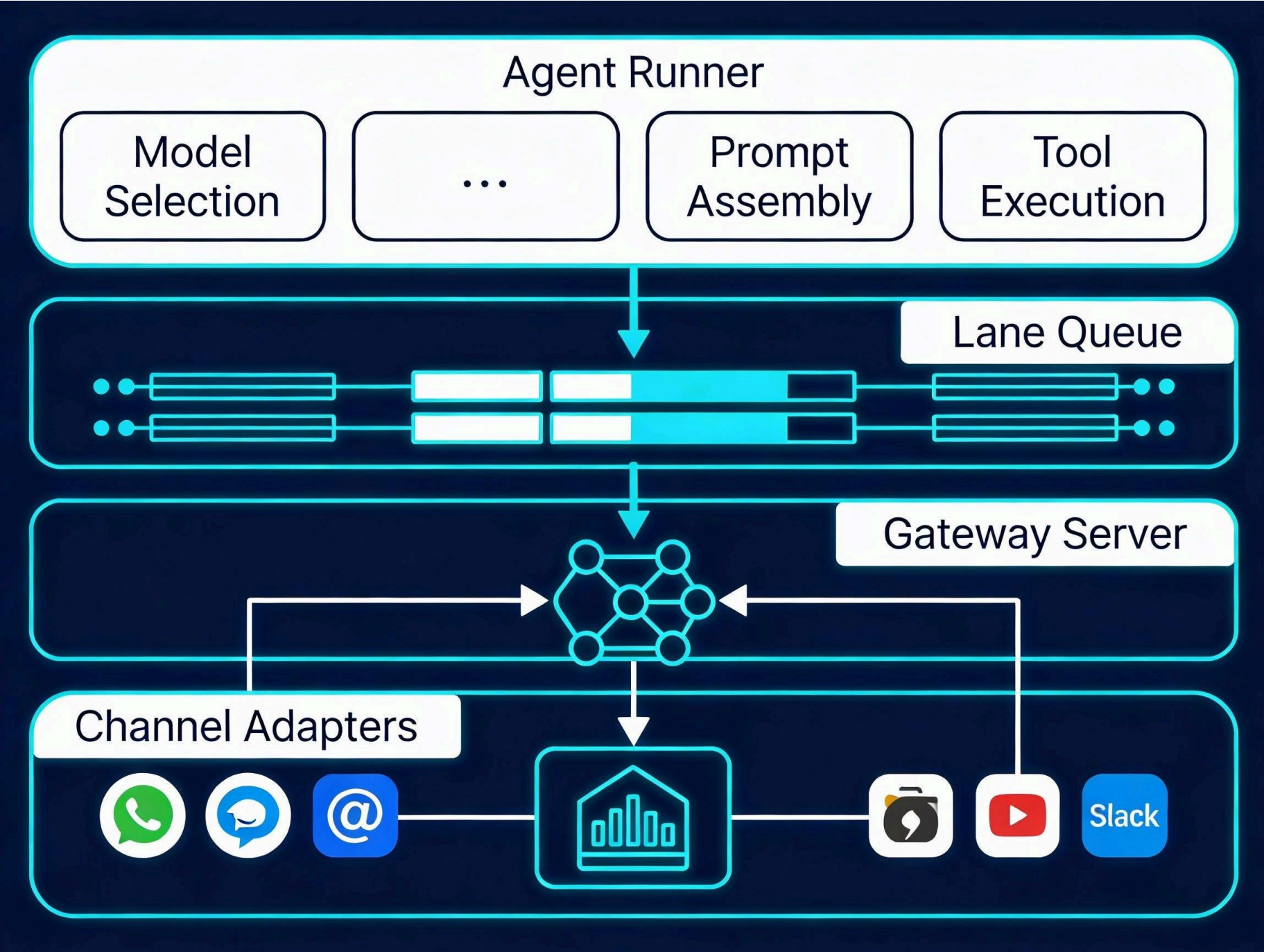
- Interface for testing tool-using AI agents via MCP servers
- Enables structured interaction between models and external tools
- Useful for debugging and prototyping agent workflows
- Bridges local tools, APIs, and LLM reasoning
- Key layer for building “tool-augmented” research systems



N8N

AUTOMATION & WORKFLOW ORCHESTRATION

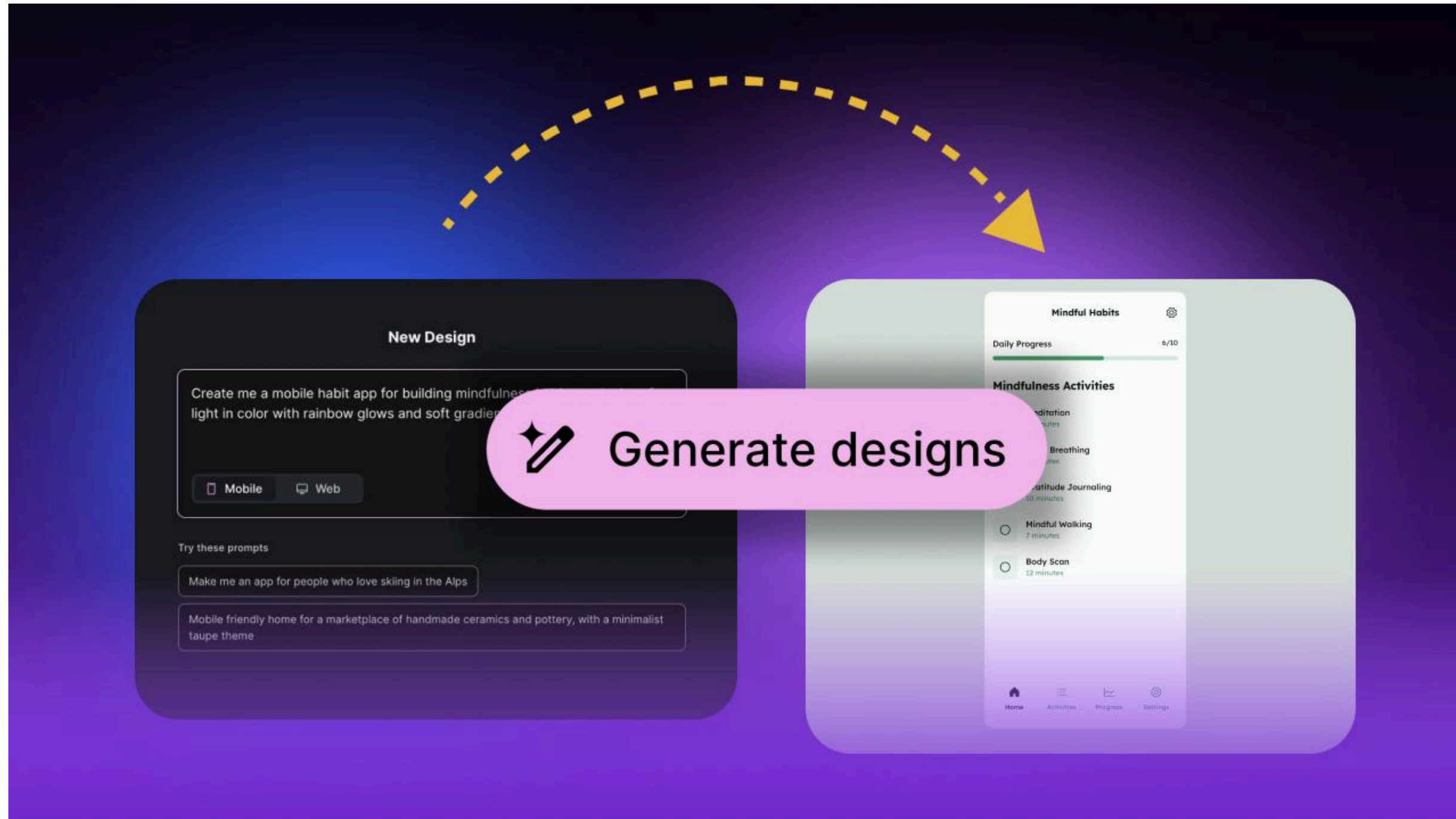
- Visual workflow automation for multi-step processes
- Connects APIs, databases, and AI tools into pipelines
- Useful for data collection and repetitive research tasks
- Strong for production workflows, weaker for rapid prototyping
- MCP integration allows AI-driven workflow generation (emerging use)



OPENCLAW

PERSONAL AGENTIC ASSISTANTS

- General-purpose personal AI assistant / agent framework
- More flexible and higher-level than CLI coding tools
- Can integrate multiple tools, memory, and workflows
- Less focused on raw coding, more on orchestration of tasks
- Bridges chat-based interaction and automated tool execution



GOOGLE STITCH

VIBE DESIGN & RAPID PROTOTYPING TOOLS

- AI-driven UI and app design generation tools
- Useful for quickly visualizing research outputs or prototypes
- Can generate posters, dashboards, or interface mockups
- Helps translate abstract ideas into visual artifacts
- Strong for communication and presentation preparation

KEY TAKEAWAY

- Most tools fall into 4 layers:
 - **Discovery** → Perplexity, Deep Research, ResearchRabbit
 - **Understanding/Visualizing** → NotebookLM, Google Stitch
 - **Execution** → Claude Code, Copilot CLI
 - **Automation & Systems** → n8n, MCP tools
- Effective research workflows often combine multiple layers
- Modern research advantage = *orchestrating tools, not mastering one*

CODING AGENTS